

Лабораторная работа № 1

Подготовка лабораторного стенда

Славинский Владислав Вадимович

Содержание

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
3.1	Подготовка программного обеспечения и рабочего каталога .	7
3.2	Настройка сценариев автоматизации	9
3.3	Формирование и регистрация box-файла	14
3.4	Запуск и проверка виртуальных машин	16
3.5	Изменение внутреннего окружения	19
4	Выводы	25
5	Ответы на контрольные вопросы	26

Список иллюстраций

3.1	Проверка установленного программного обеспечения	8
3.2	Рабочие каталоги packer и vagrant	8
3.3	Содержимое каталога packer	9
3.4	Структура каталога vagrant	9
3.5	Сценарий создания пользователя	11
3.6	Сценарий изменения имени хоста	12
3.7	Сценарий настройки маршрутизации сервера	13
3.8	Сценарий настройки сетевых интерфейсов клиента	14
3.9	Инициализация плагинов Packer	15
3.10	Сборка образа Rocky Linux	15
3.11	Сформированный box-файл	16
3.12	Регистрация box-файла в Vagrant	16
3.13	Запуск виртуальной машины server	17
3.14	Запуск виртуальной машины client	17
3.15	Подключение к серверу по SSH	18
3.16	Остановка виртуальных машин	19
3.17	Общие настройки provisioning в Vagrantfile	20
3.18	Применение сценариев настройки сервера	21
3.19	Применение сценариев настройки клиента	21
3.20	Проверка пользователя и имени сервера	22
3.21	Проверка пользователя и имени клиента	23
3.22	Завершение работы лабораторного стенда	23

Список таблиц

1 Цель работы

Приобрести практические навыки автоматизированной установки Rocky Linux на виртуальную машину с помощью Packer и управления лабораторным стендом средствами Vagrant и VirtualBox.

2 Задание

1. Сформировать box-файл с дистрибутивом Rocky Linux для VirtualBox.
2. Запустить виртуальные машины сервера и клиента и проверить их работоспособность.
3. Добавить пользователя с правами администратора и изменить имена хостов виртуальных машин `server` и `client`.
4. Подготовить файлы проекта и box-файл для резервного копирования и последующего развёртывания стенда.

3 Выполнение лабораторной работы

3.1 Подготовка программного обеспечения и рабочего каталога

Для выполнения лабораторной работы в Windows установим программы Vagrant, Packer и файловый менеджер FAR (рис. (**fig:001?**)).

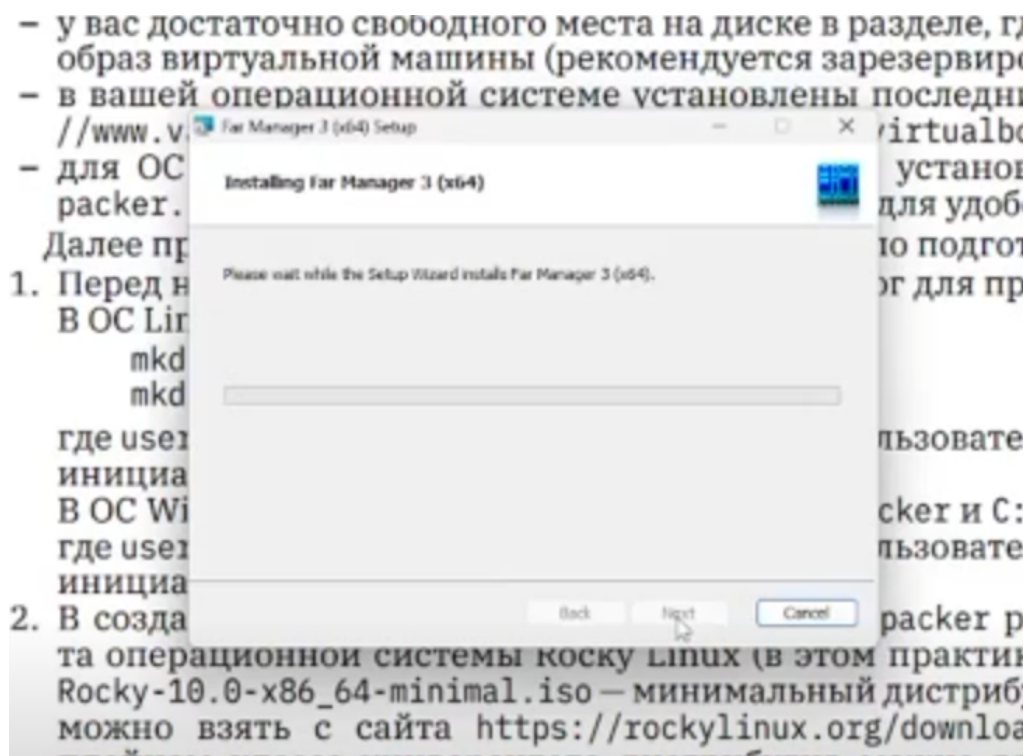


Рис. 3.1: Проверка установленного программного обеспечения

Создадим рабочие каталоги `C:\work\vvslavinskiy\packer` и `C:\work\vvslavinskiy\vagrant`. Первый каталог предназначен для автоматизированной сборки базового образа Rocky Linux, второй — для управления виртуальными машинами сервера и клиента (рис. (fig:002)).

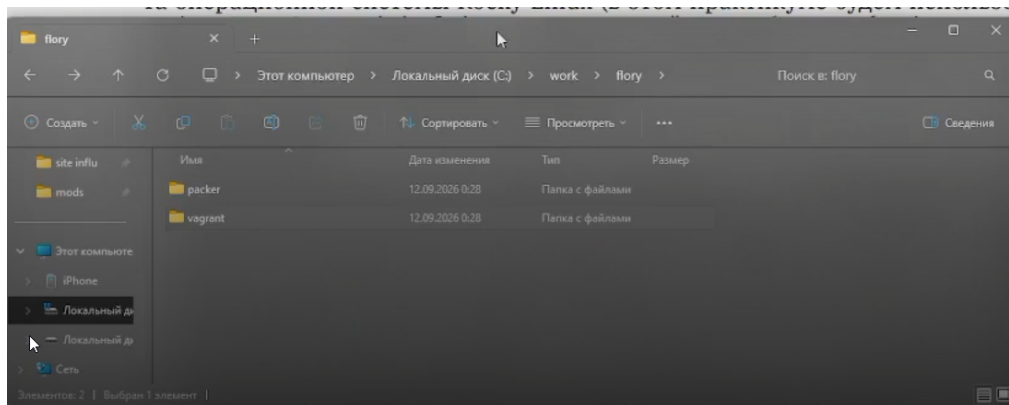


Рис. 3.2: Рабочие каталоги packer и vagrant

В каталог packer поместим установочный образ Rocky-10.2-x86_64-minimal.iso, файл конфигурации vagrant-rocky.pkr.hcl, каталог http с Kickstart-файлом rocky10-ks.cfg и вспомогательные сценарии (рис. (fig:003?)).

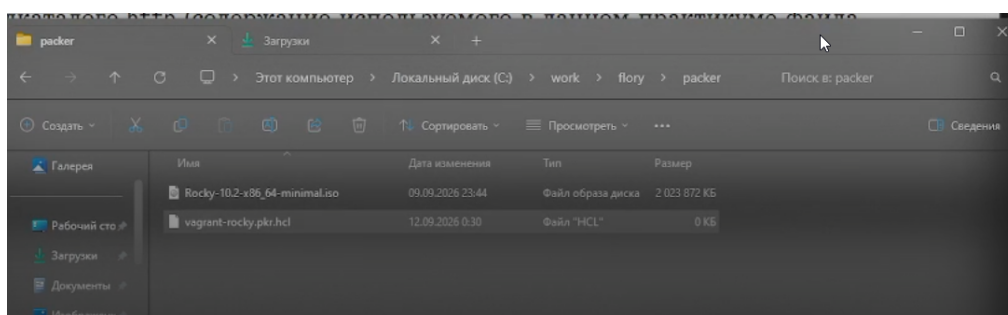


Рис. 3.3: Содержимое каталога packer

В каталоге vagrant разместим файлы Vagrantfile и Makefile. Создадим каталог provision с подкаталогами default, server и client для сценариев настройки виртуальных машин (рис. (fig:004?)).

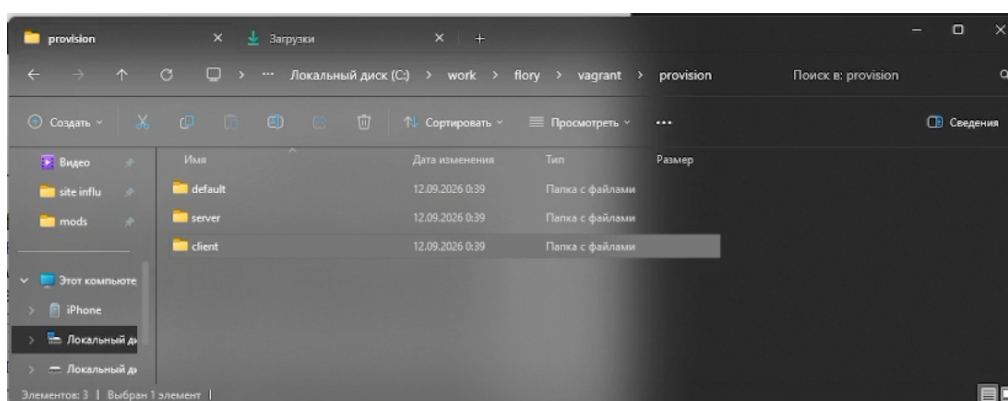


Рис. 3.4: Структура каталога vagrant

3.2 Настройка сценариев автоматизации

В файле provision/default/01-user.sh укажем имя пользователя vvslavinskiy. Сценарий создаёт пользователя, назначает ему пароль

и включает его в административную группу wheel (рис. (**fig:005?**)).

```
#!/bin/bash
```

```
echo "Provisioning script $0"
```

```
username=vvslavinskiy
```

```
userpassword=123456
```

```
encpassword=$(openssl passwd -6 ${userpassword})
```

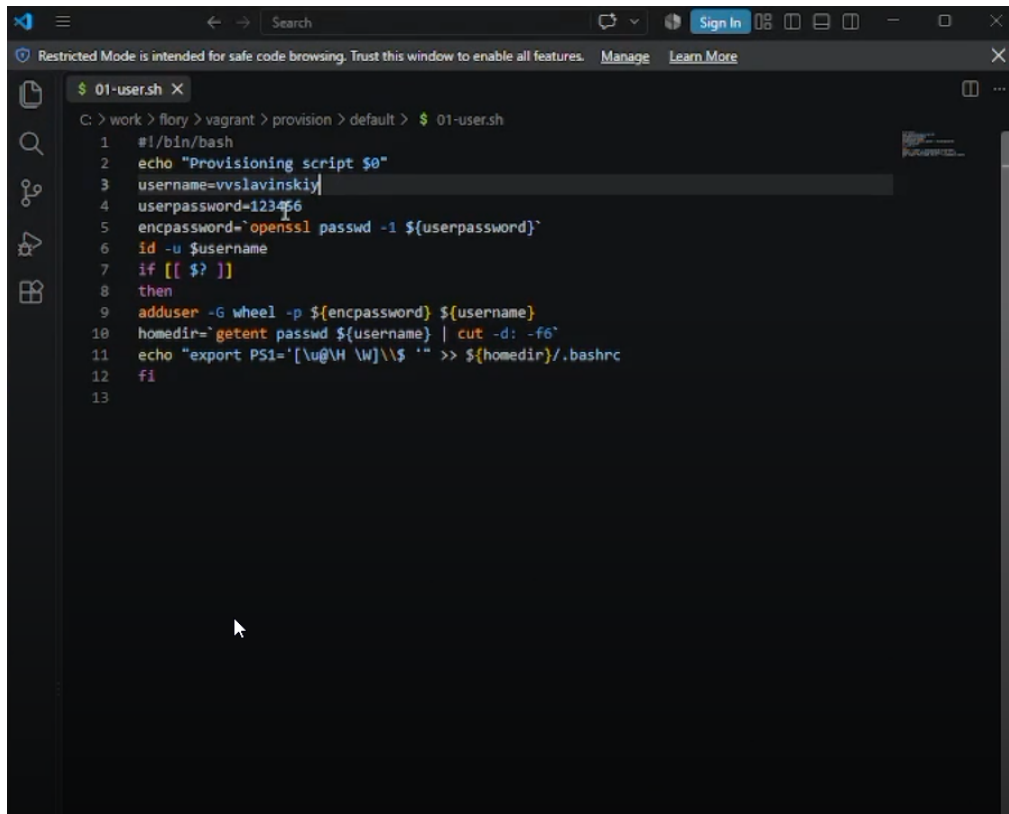
```
if ! id -u "$username" >/dev/null 2>&1; then
```

```
    adduser -G wheel -p ${encpassword} ${username}
```

```
    homedir=$(getent passwd ${username} | cut -d: -f6)
```

```
    echo "export PS1='[\u@\H \W]\\$ '" >>${homedir}/.bashrc
```

```
fi
```



```
01-user.sh X
C: > work > flory > vagrant > provision > default > $ 01-user.sh
1  #!/bin/bash
2  echo "Provisioning script $0"
3  username=vvslavinskiy
4  userpassword=123456
5  encpassword=`openssl passwd -1 ${userpassword}`
6  id -u $username
7  if [[ $? ]]
8  then
9  adduser -G wheel -p ${encpassword} ${username}
10 homedir=`getent passwd ${username} | cut -d: -f6`
11 echo "export PS1='\u@\H \W\\$ '" >> ${homedir}/.bashrc
12 fi
13
```

Рис. 3.5: Сценарий создания пользователя

В файле `provision/default/01-hostname.sh` укажем тот же идентификатор пользователя. Сценарий формирует полные имена хостов `server.vvslavinskiy.net` и `client.vvslavinskiy.net` (рис. (fig:006?)).

```
#!/bin/bash
```

```
username=vvslavinskiy
```

```
hostnamectl set-hostname "${HOSTNAME%%.*}}.${username}.net
```

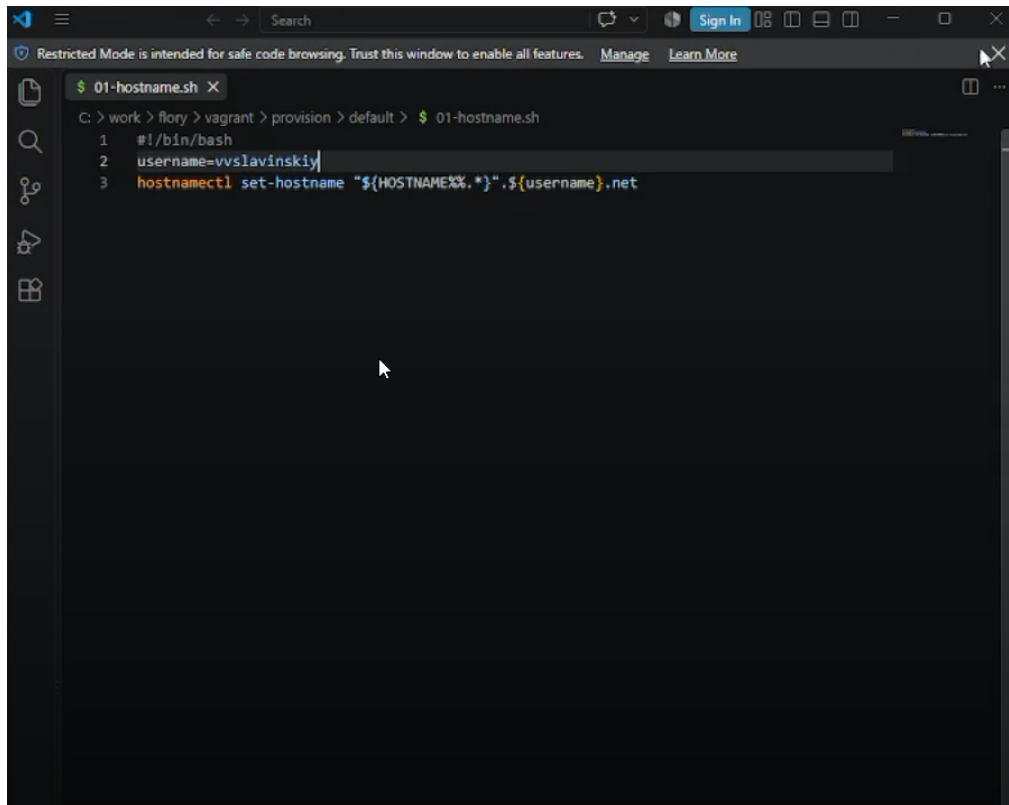


Рис. 3.6: Сценарий изменения имени хоста

В каталоге `server` разместим сценарий `02-forward.sh`, включающий пересылку IPv4-пакетов и маскердинг. Эти настройки позволяют серверу выполнять функции маршрутизатора (рис. (fig:007?)).

```
#!/bin/bash
```

```
echo "Provisioning script $0"
```

```
echo "Enable forwarding"
```

```
echo "net.ipv4.ip_forward = 1" > /etc/sysctl.d/90-forward.conf
```

```
sysctl -w net.ipv4.ip_forward=1
```

```
echo "Configure masquerading"
```

```
firewall-cmd --add-masquerade --permanent
```

```
firewall-cmd --reload
```

```
restorecon -vR /etc
```

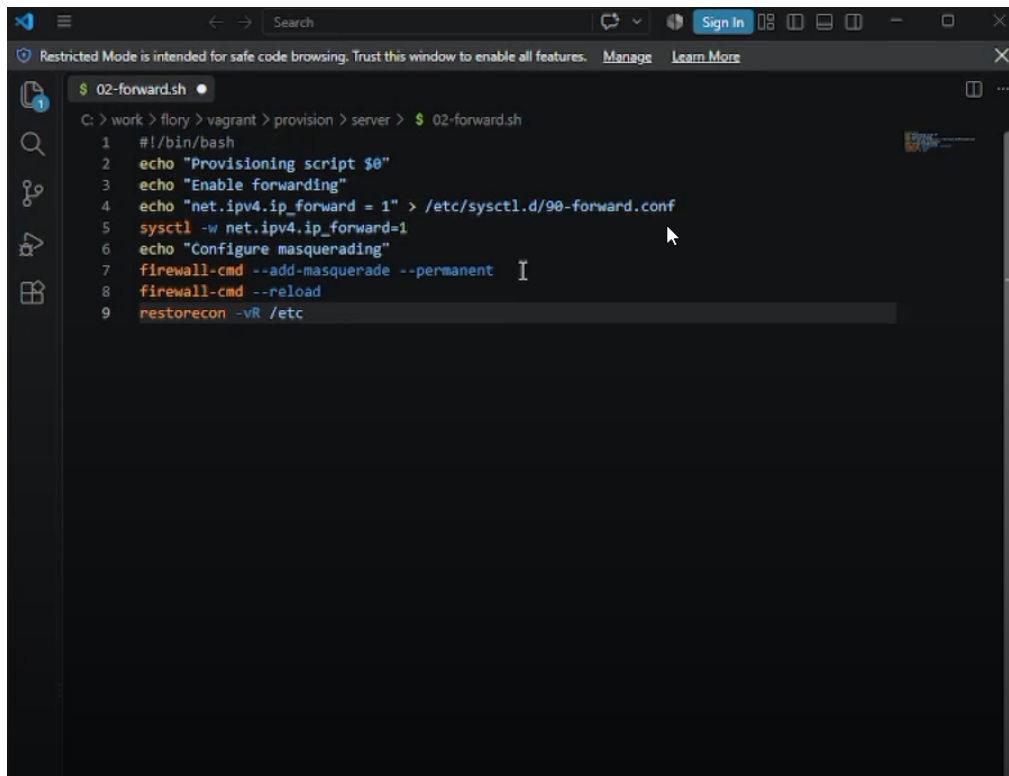


Рис. 3.7: Сценарий настройки маршрутизации сервера

В каталоге client разместим сценарий 01-routing.sh. Он настраивает внутренний интерфейс eth1, а для внешнего интерфейса eth0 запрещает использование маршрута по умолчанию (рис. (fig:008?)).

```
#!/bin/bash
```

```
echo "Provisioning script $0"
```

```
nmcli -t -f NAME connection show | grep -qx eth1 || \
```

```
nmcli connection add type ethernet ifname eth1 con-name eth1
```

```
nmcli connection modify eth1 ipv4.method auto ipv6.method link-local
```

```
nmcli connection up eth1
```

```
nmcli connection modify eth0 ipv4.never-default yes ipv6.never-default yes
```

```
nmcli device reapply eth0 || {
```

```
    nmcli connection down eth0
```

```
    nmcli connection up eth0
```

```
}
```

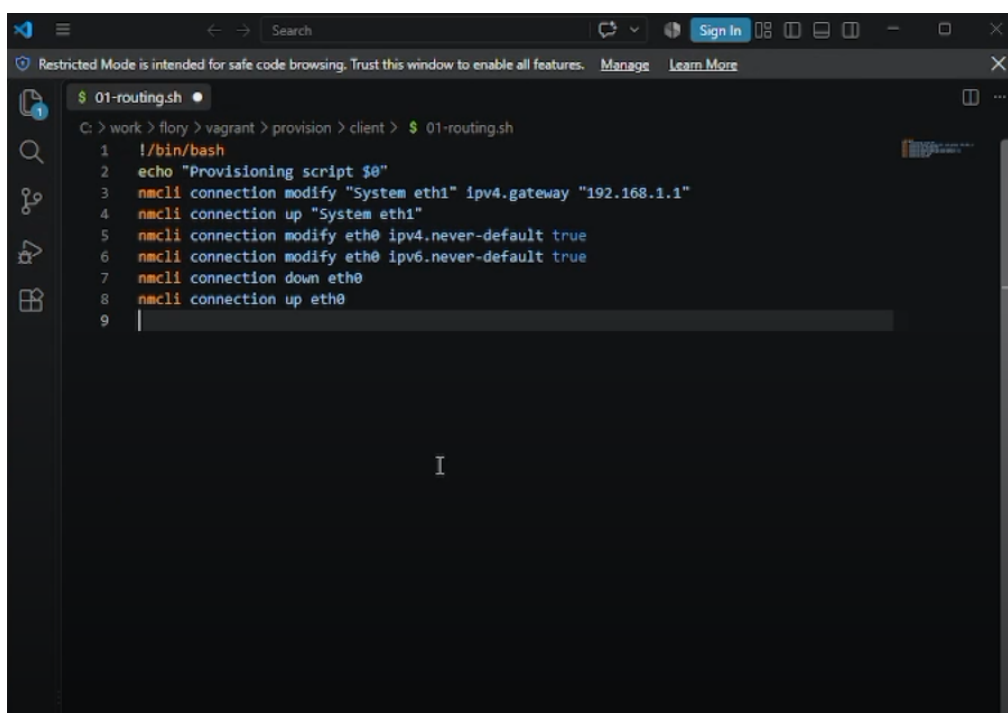


Рис. 3.8: Сценарий настройки сетевых интерфейсов клиента

3.3 Формирование и регистрация box-файла

Перейдём в каталог `C:\work\vvslavinskiy\packer` и инициализируем необходимые плагины Packer командой (рис. (fig:009?)):

```
packer.exe init vagrant-rocky.pkr.hcl
```

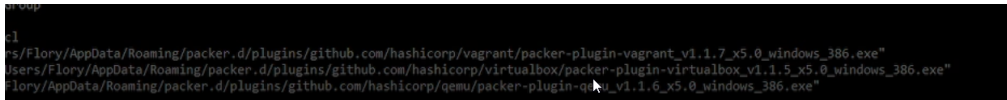


Рис. 3.9: Инициализация плагинов Packer

Запустим автоматическую установку Rocky Linux и формирование box-файла (рис. (fig:010?)):

`packer.exe build -only=virtualbox-iso.rockylinux vagrant-rocky.pkr.hcl`

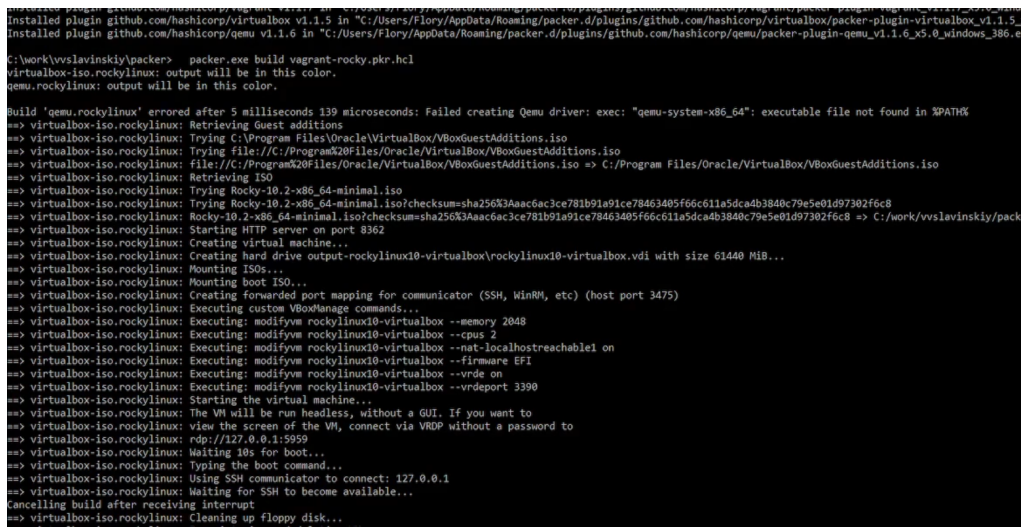


Рис. 3.10: Сборка образа Rocky Linux

После завершения сборки в рабочем каталоге появился файл `vagrant-virtualbox-rockylinux10-x86_64.box` (рис. (fig:011?)).

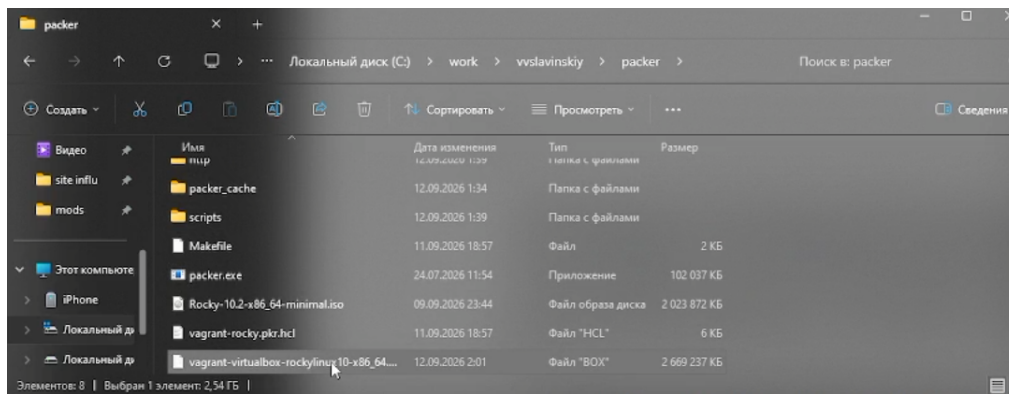


Рис. 3.11: Сформированный box-файл

Скопируем box-файл в каталог `vagrant` и зарегистрируем его под именем `rockylinux10` (рис. (fig:012?)):

```
vagrant box add rockylinux10 vagrant-virtualbox-rockylinux10-x86_64.box
```

```
C:\work\vvslavinskiy\packer> packer.exe build vagrant-rocky.pkr.hcl

C:\work\vvslavinskiy\packer>vagrant box add rockylinux10 vagrant-virtualbox-rockylinux10-x86_64.box
==> box: Box file was not detected as metadata. Adding it directly...
==> box: Adding box 'rockylinux10' (v0) for provider: (amd64)
box: Unpacking necessary files from: file:///C:/work/vvslavinskiy/packer/vagrant-virtualbox-rockylinux10-x86_64.box
box:
==> box: Successfully added box 'rockylinux10' (v0) for '(amd64)!'

C:\work\vvslavinskiy\packer>vagrant up server
A Vagrant environment or target machine is required to run this
command. Run 'vagrant init' to create a new Vagrant environment. Or,
get an ID of a target machine from 'vagrant global-status' to run
this command on. A final option is to change to a directory with a
```

Рис. 3.12: Регистрация box-файла в Vagrant

3.4 Запуск и проверка виртуальных машин

Из каталога `C:\work\vvslavinskiy\packer` запустим виртуальную машину сервера (рис. (fig:013?)):

```
vagrant up server
```

```

==> box: Successfully added box 'rockylinux10' (v0) for '(amd64)' !

C:\work\vvslavinskiy\packer>vagrant up server
A Vagrant environment or target machine is required to run this
command. Run `vagrant init` to create a new Vagrant environment. Or,
get an ID of a target machine from `vagrant global-status` to run
this command on. A final option is to change to a directory with a
Vagrantfile and to try again.

```

Рис. 3.13: Запуск виртуальной машины server

Аналогично запустим виртуальную машину клиента (рис. (fig:014?)):

vagrant up client

```

C:\work\vvslavinskiy\vagrant>vagrant up client
Bringing machine 'client' up with 'virtualbox' provider...
==> client: Preparing master VM for linked clones...
    client: This is a one time operation. Once the master VM is prepared,
    client: it will be used as a base for linked clones, making the creation
    client: of new VMs take milliseconds on a modern system.
==> client: Importing base box 'rockylinux10'...
==> client: Cloning VM...
==> client: Matching MAC address for NAT networking...
==> client: Setting the name of the VM: vagrant_client_1789169490386_66718
Vagrant is currently configured to create VirtualBox synced folders with
the `sharedFolders.enabled` option enabled. If the Vagrant

```

Рис. 3.14: Запуск виртуальной машины client

Подключимся к серверу по SSH и проверим возможность входа в систему (рис. (fig:015?)):

vagrant ssh server

```

server: Running: C:/Users/Flory/AppData/Local/Temp/vagrant-shell20260912-
6-z41yq9.sh
server: Provisioning script /tmp/vagrant-shell

C:\work\vvslavinskiy\vagrant>vagrant ssh server
==> server: The machine you're attempting to SSH into is configured to use
==> server: password-based authentication. Vagrant can't script entering the
==> server: password for you. If you're prompted for a password, please enter
==> server: the same password you have configured in the Vagrantfile.
vagrant@127.0.0.1's password:
vagrant@127.0.0.1's password:
vagrant@127.0.0.1's password:
Last failed login: Fri Sep 11 23:43:46 UTC 2026 from 10.0.2.2 on ssh:notty
There were 2 failed login attempts since the last successful login.
Last login: Fri Sep 11 23:42:48 2026
vagrant@server:~$ su - user
su: user user does not exist or the user entry does not contain all the requi
fields
vagrant@server:~$ su - vvslavinskiy
Password:
[vvslavinskiy@server.vvslavinskiy.net ~]$ exit
logout
vagrant@server:~$

```

Рис. 3.15: Подключение к серверу по SSH

После первичной проверки завершим SSH-сеанс и остановим обе виртуальные машины:

```
vagrant halt server
```

```
vagrant halt client
```

Результат корректной остановки машин представлен на рис. (**fig:016?**).

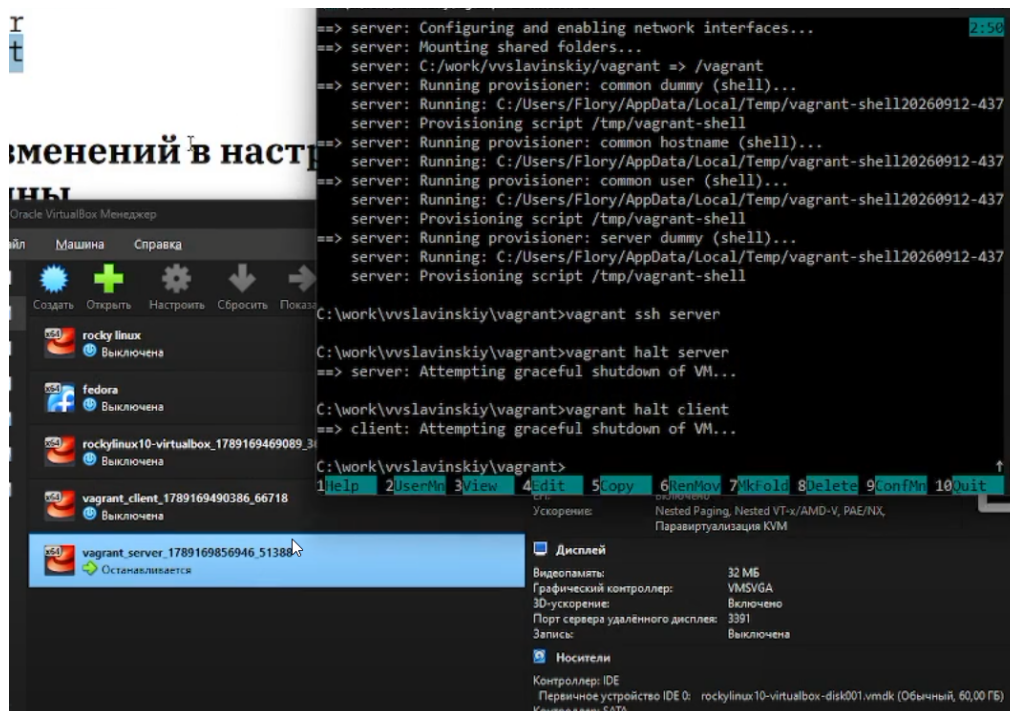


Рис. 3.16: Остановка виртуальных машин

3.5 Изменение внутреннего окружения

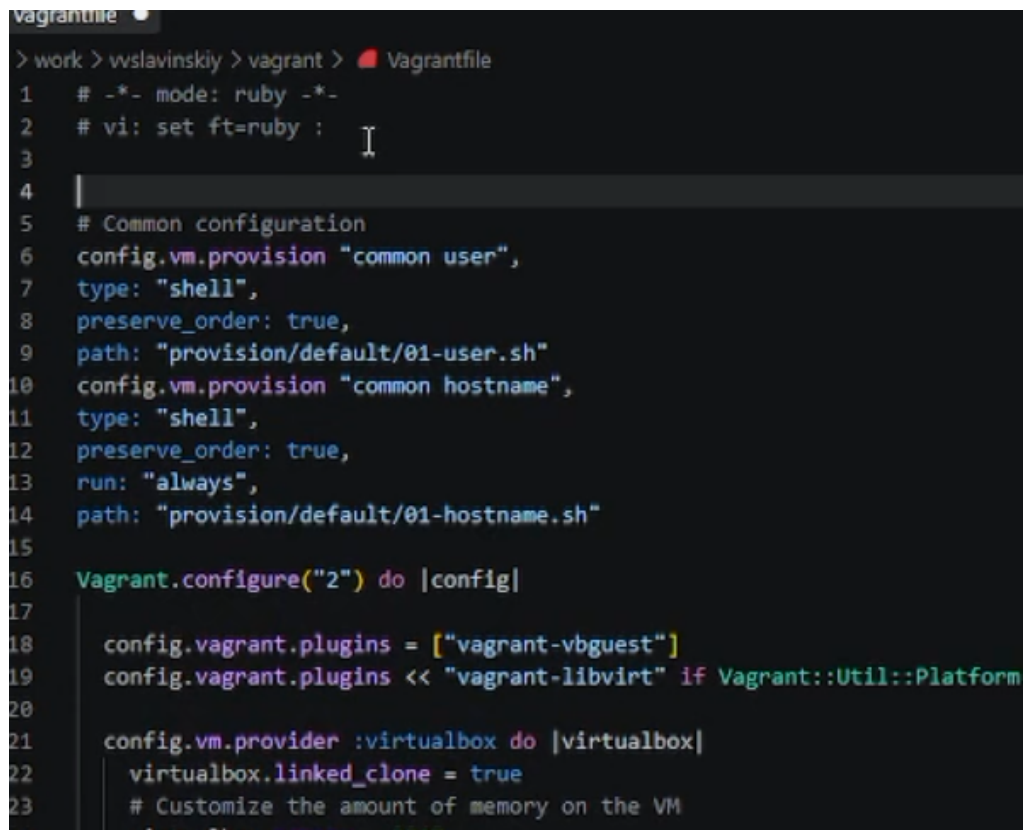
Добавим в общую конфигурацию Vagrantfile вызовы сценариев создания пользователя и изменения имени хоста:

```
config.vm.provision "common hostname",
    type: "shell",
    preserve_order: true,
    run: "always",
    path: "provision/default/01-hostname.sh"
```

```
config.vm.provision "common user",
    type: "shell",
    preserve_order: true,
```

`path: "provision/default/01-user.sh"`

Внесённые изменения в Vagrantfile показаны на рис. (fig:017?).

A screenshot of a text editor showing a Vagrantfile. The file is opened in a window titled 'vagrantfile'. The terminal shows the path '> work > wslavinskiy > vagrant > Vagrantfile'. The code is as follows:

```
1  # -*- mode: ruby -*-
2  # vi: set ft=ruby :
3
4
5  # Common configuration
6  config.vm.provision "common user",
7  type: "shell",
8  preserve_order: true,
9  path: "provision/default/01-user.sh"
10 config.vm.provision "common hostname",
11 type: "shell",
12 preserve_order: true,
13 run: "always",
14 path: "provision/default/01-hostname.sh"
15
16 Vagrant.configure("2") do |config|
17
18   config.vagrant.plugins = ["vagrant-vbguest"]
19   config.vagrant.plugins << "vagrant-libvirt" if Vagrant::Util::Platform
20
21   config.vm.provider :virtualbox do |virtualbox|
22     virtualbox.linked_clone = true
23     # Customize the amount of memory on the VM
```

Рис. 3.17: Общие настройки provisioning в Vagrantfile

Применим сценарии настройки сначала к серверу, затем к клиенту (рис. (fig:018?), (fig:019?)):

```
vagrant up server --provision
```

```
vagrant up client --provision
```

```

Path: C:/work/vvslavinskiy/vagrant/Vagrantfile
line number: 6
Message: NameError: undefined local variable or method `config' for main

C:\work\vvslavinskiy\vagrant>vagrant up server --provision
Bringing machine 'server' up with 'virtualbox' provider...
==> server: You assigned a static IP ending in ".1" or ":1" to this machine
==> server: This is very often used by the router and can cause the
==> server: network to not work properly. If the network doesn't work
==> server: properly, try changing this IP.
==> server: You assigned a static IP ending in ".1" or ":1" to this machine
==> server: This is very often used by the router and can cause the
==> server: network to not work properly. If the network doesn't work
==> server: properly, try changing this IP.
==> server: Clearing any previously set forwarded ports...
==> server: Clearing any previously set network interfaces...
==> server: Preparing network interfaces based on configuration...
server: Adapter 1: nat
server: Adapter 2: intnet
==> server: Forwarding ports...
server: 22 (guest) => 2222 (host) (adapter 1)
==> server: Running 'pre-boot' VM customizations...

```

Рис. 3.18: Применение сценариев настройки сервера

```

server: Running: C:/Users/Flory/AppData/Local/Temp/vagrant-shell2026091
08-y56k4c.sh
server: Provisioning script /tmp/vagrant-shell

C:\work\vvslavinskiy\vagrant>vagrant up client --provision
Bringing machine 'client' up with 'virtualbox' provider...
==> client: Clearing any previously set forwarded ports...
==> client: Fixed port collision for 22 => 2222. Now on port 2200.
==> client: Clearing any previously set network interfaces...
==> client: Preparing network interfaces based on configuration...
client: Adapter 1: nat
client: Adapter 2: intnet
==> client: Forwarding ports...
client: 22 (guest) => 2200 (host) (adapter 1)
==> client: Running 'pre-boot' VM customizations...
==> client: Booting VM...

```

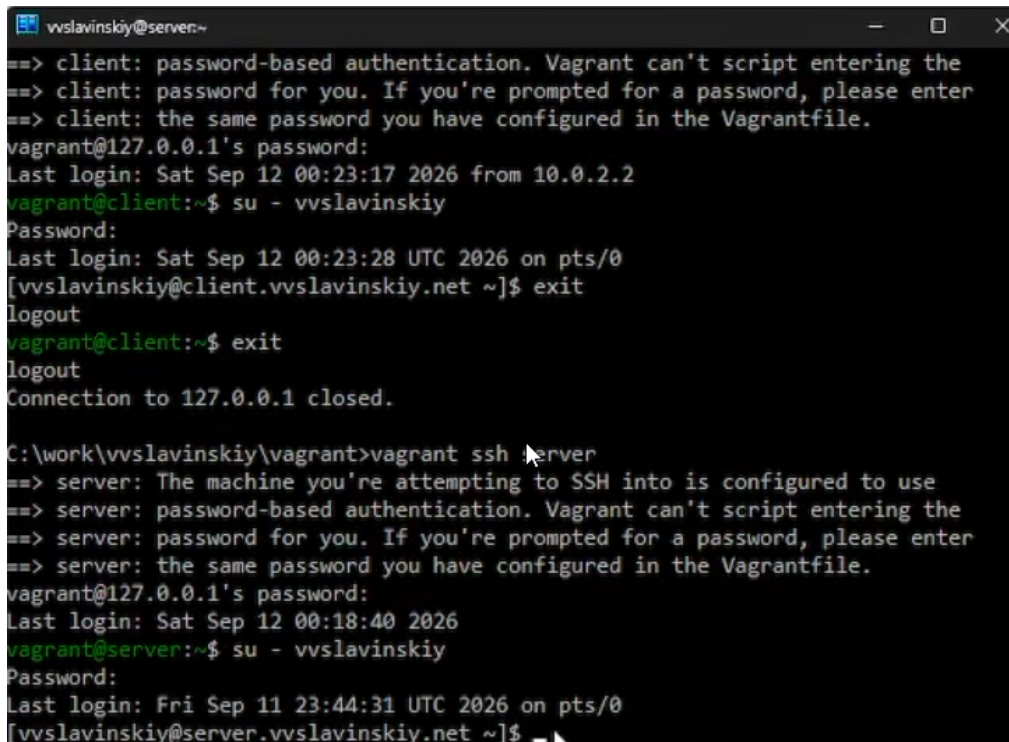
Рис. 3.19: Применение сценариев настройки клиента

Подключимся к серверу и перейдём к созданному пользователю:

```
vagrant ssh server
```

```
su - vvslavinskiy
hostname
id
```

Приглашение командной строки и команда `hostname` подтверждают, что пользователь создан, имеет необходимые права, а сервер получил имя `server.vvslavinskiy.net` (рис. (fig:020?)).



```
vvslavinskiy@server:~$
==> client: password-based authentication. Vagrant can't script entering the
==> client: password for you. If you're prompted for a password, please enter
==> client: the same password you have configured in the Vagrantfile.
vagrant@127.0.0.1's password:
Last login: Sat Sep 12 00:23:17 2026 from 10.0.2.2
vagrant@client:~$ su - vvslavinskiy
Password:
Last login: Sat Sep 12 00:23:28 UTC 2026 on pts/0
[vvslavinskiy@client.vvslavinskiy.net ~]$ exit
logout
vagrant@client:~$ exit
logout
Connection to 127.0.0.1 closed.

C:\work\vvslavinskiy\vagrant>vagrant ssh server
==> server: The machine you're attempting to SSH into is configured to use
==> server: password-based authentication. Vagrant can't script entering the
==> server: password for you. If you're prompted for a password, please enter
==> server: the same password you have configured in the Vagrantfile.
vagrant@127.0.0.1's password:
Last login: Sat Sep 12 00:18:40 2026
vagrant@server:~$ su - vvslavinskiy
Password:
Last login: Fri Sep 11 23:44:31 UTC 2026 on pts/0
[vvslavinskiy@server.vvslavinskiy.net ~]$
```

Рис. 3.20: Проверка пользователя и имени сервера

Аналогично подключимся к клиенту:

```
vagrant ssh client
su - vvslavinskiy
hostname
id
```

В результате приглашение командной строки содержит имя `client.vvslavinskiy.net`, что подтверждает корректную настройку клиента (рис. (fig:021?)).

```
vvslavinskiy@server:~$  
==> client: password-based authentication. Vagrant can't script entering the  
==> client: password for you. If you're prompted for a password, please enter  
==> client: the same password you have configured in the Vagrantfile.  
vagrant@127.0.0.1's password:  
Last login: Sat Sep 12 00:23:17 2026 from 10.0.2.2  
vagrant@client:~$ su - vvslavinskiy  
Password:  
Last login: Sat Sep 12 00:23:28 UTC 2026 on pts/0  
[vvslavinskiy@client.vvslavinskiy.net ~]$ exit  
logout  
vagrant@client:~$ exit  
logout  
Connection to 127.0.0.1 closed.
```

Рис. 3.21: Проверка пользователя и имени клиента

После проверки выйдем из SSH-сеансов и выключим обе виртуальные машины (рис. (fig:022?)):

```
vagrant halt client
```

```
vagrant halt server
```

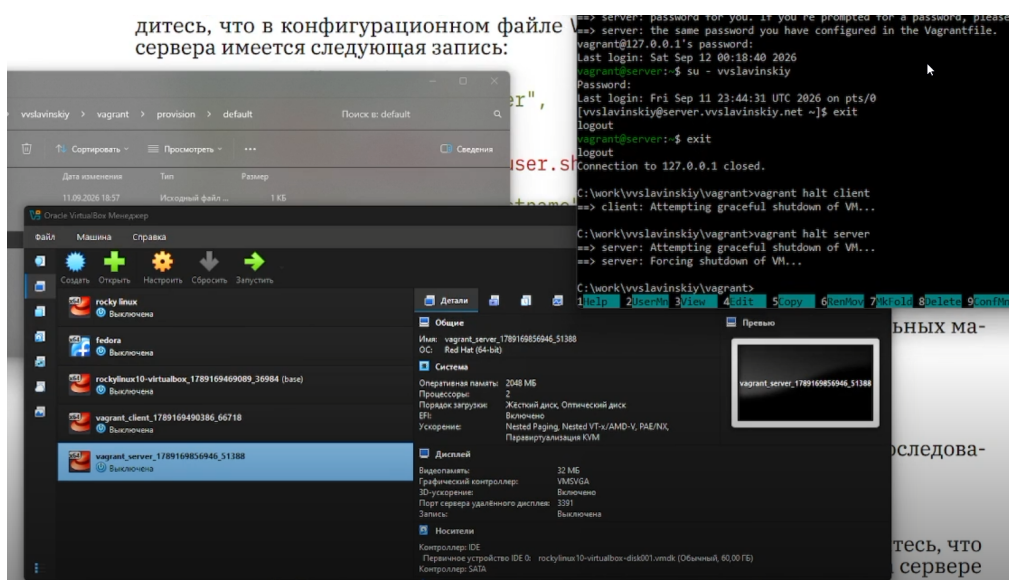


Рис. 3.22: Завершение работы лабораторного стенда

Скопируем файлы `vagrant-virtualbox-rockylinux10-x86_64.box`, `vagrant-rocky.pkr.hcl`, `rocky10-ks.cfg`, `Vagrantfile`, `Makefile` и каталог `provision` в каталог резервной копии. Эти файлы позволяют повторно развернуть лабораторный стенд на другом компьютере.

4 Выводы

В ходе выполнения лабораторной работы был сформирован box-файл Rocky Linux 10 для VirtualBox. С помощью Vagrant были развёрнуты виртуальные машины сервера и клиента. Сценариями provisioning был создан пользователь `vvslavinskiy` с административными правами, изменены имена хостов и подготовлены сетевые настройки стенда. Работоспособность обеих виртуальных машин была подтверждена подключением по SSH.

5 Ответы на контрольные вопросы

1. Для чего предназначен Vagrant?

Vagrant предназначен для автоматизированного создания, настройки, запуска и удаления воспроизводимых виртуальных окружений. Параметры окружения описываются в Vagrantfile, благодаря чему один и тот же стенд можно развернуть на разных компьютерах.

2. Что такое box-файл? В чём назначение Vagrantfile?

Box-файл представляет собой упакованный базовый образ виртуальной машины вместе с метаданными провайдера. Vagrantfile описывает, какой box-файл использовать, какие виртуальные машины создать, сколько ресурсов им выделить, как настроить сеть и какие сценарии provisioning выполнить.

3. Основные команды Vagrant.

- `vagrant box list` — показать зарегистрированные box-файлы;
- `vagrant box add NAME FILE` — зарегистрировать box-файл;
- `vagrant init` — создать начальный Vagrantfile;
- `vagrant up [VM]` — создать и запустить виртуальную машину;
- `vagrant ssh [VM]` — подключиться к машине по SSH;
- `vagrant provision [VM]` — повторно выполнить сценарии настройки;

- `vagrant reload [VM]` — перезагрузить машину с применением конфигурации;
- `vagrant halt [VM]` — корректно выключить машину;
- `vagrant destroy [VM]` — удалить созданную виртуальную машину;
- `vagrant status` — показать состояние машин.

4. Назначение конфигурационных файлов.

Файл `vagrant-rocky.pkr.hcl` задаёт источники и версии плагинов Packer, параметры Rocky Linux, контрольную сумму ISO, характеристики виртуальной машины, команды автомат boot-процесса, сценарии подготовки системы и формирование итогового `box`-файла. Файл `rocky10-ks.cfg` автоматизирует установку Rocky Linux: настраивает загрузчик, разделы, язык, клавиатуру, сеть, пользователей, SSH, службы и набор пакетов. `Vagrantfile` описывает общие параметры провайдера VirtualBox, машины `server` и `client`, внутреннюю сеть и выполняемые сценарии `provisioning`. `Makefile` объединяет типовые команды проверки провайдера, установки плагинов, регистрации `box`-файла, запуска, настройки, остановки и удаления виртуальных машин.